

Scenario Planner

Last updated: 2026-05-17

The Scenario Planner lets studios model hypothetical production schedules. The Scenario Planner lets studios model hypothetical production schedules, answering "when can we ship?" or "what if we aim for date X?", without touching their source tool. It combines a scope-gathering conversation with Claude Haiku, a high-fidelity generation engine, and an interactive viewer for the resulting product/asset/work plan.

Concepts

Session: the container for one planning run. Scoped to a user+studio pair. Only one active session per user at a time; starting a new one dismisses the previous.

Scope: the set of planning parameters that drive generation:

Parameter	Type	Description
<code>scenario_category</code>	<code>earliest_ship</code> <code>target_date</code>	Schedule forward from today (derive completion) or backward from a target date
<code>target_date</code>	date string	Required for <code>target_date</code> category, the desired ship date for the final product
<code>release_cadence</code>	enum	<code>single_launch</code> , <code>regular_releases</code> , or <code>milestone_batched</code>
<code>num_products</code>	integer	Exact number of products/releases to generate
<code>release_interval_days</code>	integer	Explicit spacing between releases (overrides horizon-derived spacing)
<code>distribution</code>	enum	How assets are spread: <code>even</code> , <code>front_loaded</code> , <code>back_loaded</code> , <code>milestone_batched</code>
<code>scale</code>	object	Exact asset count per classification profile (e.g. <code>{"Hero High": 30, "Support Low": 60}</code>)
<code>craft_caps</code>	object	Optional per-craft concurrent-asset caps (e.g. <code>{"3D": 5, "2D": 8}</code>)
<code>constraints</code>	string[]	Optional hard constraints (e.g. "no character work before March")

Classification profile: a label derived from the estimate matrix's variable value combinations (e.g. `Hero | High`). The scoping agent presents these using the studio's actual matrix profiles.

Generation mode: `ai` (Claude Sonnet multi-pass) or `rule_based` (deterministic, no AI calls). Both modes write to the same tables and produce the same output format.

Scenario category, Scenario category, `earliest_ship**`: the engine schedules forward from today. Product release dates are computed and written back to `scenario_products` after work items are placed.

Scenario category, Scenario category, `target_date**`: the engine schedules backward from the given target date. Feasibility is checked pre-generation and infeasible profiles produce preflight warnings.

Preflight warnings: conditions surfaced before generation that make a scenario unlikely to succeed: sparse matrix cells (a profile has no steps with estimates), or unreachable target dates given the critical-path length of a profile's workflow.

Session Lifecycle

```
POST /api/scenario/start
├─ Gate: estimate_matrix must exist
├─ If active session exists → return it (resume)
├─ Create new session (ai_stage = "scoping")
│   │
│   │ User sends messages via POST /api/scenario/{id}/message
│   │ Haiku asks clarifying questions, refines scope
│   │
│   │ Haiku calls submit_scope tool → ai_stage = "pending_generation"
│   │
│   │ Background loop (_scenario_generation_loop in main.py) picks up the session
│   │ Runs generation (AI or rule-based) → writes scenario_products, scenario_assets, scenario
│   │ ai_stage → "discussion"
│   │
│   │ User sends messages → discussion mode (Haiku answers questions about the generated plan)
│   │
│   │ DELETE /api/scenario/{id} → dismissed
```

Force-generate escape hatch: after turn 5, the frontend shows a "Generate with what I have" button.

`POST /api/scenario/{id}/force-generate` extracts a partial scope from the conversation history (using a Haiku tool-forced call) and immediately advances to `pending_generation`. Useful when the user has given enough context but doesn't want to answer more questions.

Retry: if generation fails, `ai_stage = "generation_failed"`. `POST /api/scenario/{id}/retry-generation` resets to `pending_generation` without re-running the scoping phase.

Overload error handling: if the Anthropic API returns a 429 (rate limit) or 529 (overloaded) response during any `send_message` call, the route catches `anthropic.APIStatusError` and returns HTTP 503 with a plain-text `detail` string. The frontend renders this as an inline error banner in the chat UI. A 400 `invalid_request_error` from the API (e.g. token limit) is not caught here and propagates as a 500.

AI Scoping (Haiku)

During the `scoping` stage, Claude Haiku is prompted with: - A full matrix summary (step × variable combination × estimate days) from the studio's data - The list of profiles (unique classification combinations with at least one estimate > 0) - The list of crafts from `workflow_steps` - The chosen `scenario_category` (shapes the questions asked)

The model has a single tool available: `submit_scope`, a structured tool call that captures all required scope fields. Once the model calls it, the session advances to `pending_generation`.

At turn 8, the route forces `tool_choice = {type: "tool", name: "submit_scope"}` regardless of whether the model would have chosen it naturally, ensuring the scoping phase always terminates.

Prompt caching (`cache_control: {type: "ephemeral"}`) is applied to the system prompt to reduce latency on subsequent turns within the same scoping conversation.

Discussion Mode (Haiku)

After generation completes (`ai_stage = "discussion"`), Haiku answers questions about the generated plan. The discussion prompt is built in `lib/scenario/context.py` (`build_discussion_prompt`) and contains four sections:

1. **Instructions + definitions:** strict grounding rules (no fabricated numbers, all derivations shown, days-not-weeks), the **asset-first scheduling philosophy** as the default lens for every recommendation (relieve a serializing craft by raising its cap / adding FTEs so assets keep flowing through their full pipeline — never recommend craft-first / department-batched scheduling that fragments asset timelines; see [Asset-at-a-time placement](#)), production planning formulas (critical path, float, utilisation, throughput), and the SCENARIO_ACTION protocol.
2. **Generation constraints:** the scope used to produce this scenario (mode, target date, cadence, scale, craft caps, distribution).
3. **Workflow structure:** step names, crafts, and the dependency DAG. Estimate values are intentionally omitted: each asset has exactly one profile with one estimate per step; summing across profiles produces meaningless totals. Per-asset estimates are available on demand via `get_asset_schedule`.
4. **Scenario data:** compact aggregated summary (product spans, asset profile distribution, per-craft cap utilisation with peak date and days-at/above/below-cap, work item counts by step). Per-asset and per-product detail is fetched on demand via tools.

Discussion tool loop

Haiku may call multiple tools in a single response. The handler (`_handle_discussion` in `routes/scenario.py`) runs up to 5 rounds:

- Each round: Haiku responds → tool blocks extracted → all tools executed **in parallel** via `asyncio.gather` → results fed back as a single `tool_result` user message.
- If a round produces no tool blocks, the text response is captured and the loop exits.

- If all 5 rounds consume tool calls without producing a text response, a forced final text request is issued (tool-free) to ensure a reply is always returned.

Discussion tools

Tool	Description
<code>get_craft_overrun_assets</code>	Returns assets whose work overlaps above-cap periods for a given craft. Uses a sweep-line event sort (end events before start events on the same date) to avoid false positives at batch boundaries. Returns the empty set if no cap was set.
<code>list_assets_in_product</code>	Returns the full asset roster for a named product, names, profiles, priorities. Accepts case-insensitive partial match on product name.
<code>get_asset_schedule</code>	Returns the step-by-step work schedule for a single named asset, step, craft, start/end dates, estimate days. Accepts case-insensitive partial match on asset name.

SCENARIO_ACTION: iterating on a scenario

When a user explicitly asks to change scope parameters and regenerate, Haiku appends a structured `SCENARIO_ACTION` block to its response. The route parses this block and returns it as `action` in the message response.

```
SCENARIO_ACTION: {"type":"regenerate","scope_changes":{"..."},"description":"..."}

```

Valid `scope_changes` keys:

Key	Type	Description
<code>craft_caps</code>	object	Craft name → integer cap. <code>null</code> removes a cap. Only include crafts the user explicitly changed.
<code>release_interval_days</code>	integer	New cadence interval in days
<code>num_products</code>	integer	New number of sprints/releases
<code>scale</code>	object	Profile label → integer count. Only include profiles the user changed.

The frontend renders an `ActionCard` component when `action.type ≡ "regenerate"`. The user can **Apply & Regenerate** (calls `POST /api/scenario/{id}/regenerate` with `scope_changes`, which deep-merges the changes into the stored scope and re-queues generation) or **Dismiss**. Haiku only emits `SCENARIO_ACTION` for explicit "change and re-run" requests, not for hypothetical "what if" questions.

Generation Engines

Two engines, same output tables.

Rule-based engine (`lib/scenario/deterministic.py`)

No AI calls. Every planning decision is a pure function of `(scope, matrix, workflow_graph)`. Fully reproducible given the same inputs.

Entry point: `run_rule_based_generation(session_id, studio_id, scope)`

Step 1, Products

Generates `num_products` records with cadence-aware names and release dates spaced by `release_interval_days` (or derived from `horizon_months` / `target_date`):

- `single_launch` → "Launch"
- `regular_releases`, interval ≥ 80 days → quarterly labels ("Q3 '26")
- `regular_releases`, interval ≥ 25 days → monthly labels ("Jun '26")
- `regular_releases`, shorter → "Sprint N"
- `milestone_batched`, $n=3$ → "Alpha / Beta / Gold"; $n=4$ → adds "Live"

For `earliest_ship` with no interval, product dates are left `NULL` and written back after scheduling.

Step 2, Asset profiles

Maps each `scope.scale` label (e.g. "Hero | High") to its matrix combo key, reconstructs `variable_values` from the key, and validates against known combo keys. A count normalisation pass corrects rounding drift so total asset count matches exactly.

Step 2.5, Preflight checks

Two checks before any assets or work are written:

- **Matrix sparsity:** if a requested profile has no steps with `estimate_days > 0`, a warning is appended to `scenario_sessions.preflight_warnings`.
- **Target date feasibility:** for `target_date` mode, the critical path length of each profile's workflow DAG is computed and compared against the available days. Profiles whose critical path exceeds the window produce an `infeasible_timeline` warning with day counts.

Step 2b, Asset expansion

Assets are distributed across products according to `scope.distribution` (even / front-loaded / back-loaded / milestone-batched) and inserted in batches of 500.

Step 3, Work templates

One template is built per distinct primary-variable value (the first `variable_field`). Each template lists the workflow steps that have a non-zero estimate for a representative combo key from that group, sorted in topological order.

Step 4, Work scheduling

Two paths based on whether `release_interval_days` is set in an `earliest_ship` scenario:

- **Single-launch** (`expand_and_insert_work`), processes one product at a time; each asset's work block is placed relative to that product's release date (backward) or a rolling cursor (forward).
 - **Cadence** (`expand_and_insert_work_cadence`), forward-schedules all assets in a single global pass; sprint release dates are then derived by working backwards from the last sprint's actual completion date, keeping the requested interval unless a sprint slips.
-

Scheduling Algorithm

Asset-at-a-time placement

The scheduler processes one complete asset at a time, placing all of an asset's workflow steps before moving on to the next asset. This is a deliberate design choice aligned with the PAW principle: the asset is the most important entity, and its work is a property of that asset's journey through the pipeline. Step-level interleaving (classic RCPSP) would pack craft capacity more tightly but would fragment individual asset timelines into disconnected bursts, which is operationally unworkable for art teams.

Uncapped scheduling

Backward (target_date): terminal steps (no successors) are anchored to the product release date. Each predecessor's end date is `min(successor.start) - 1`. The DAG is traversed in reverse topological order.

Forward (earliest_ship): each step starts at `max(deps.finish) + 1`, or the product cursor if it has no dependencies. Independent branches execute in parallel.

Uncapped schedule cache: within a single generation run, all assets sharing the same (`combo_key`, `product_id`) pair receive identical dates (same steps, same anchor). The result is computed once and cloned, skipping redundant DAG traversal for duplicate asset types within the same product.

Capped scheduling

When `craft_caps` are set, the scheduler uses a two-pass approach:

Pass 1, forward cap-push: the chain starts at `_batch_chain_start`, which pre-computes how far back the product window must open to fit all assets given their total cap-constrained workload. Within that window, steps are forward-scheduled with cap enforcement. Each capped step's start date is pushed forward until the craft's concurrent-asset count drops below its cap.

Pass 2, backward pullback (backward mode only): uncapped steps (prep, polish, review) are pulled backwards to sit just before their successors' actual start dates. Only steps with zero capped ancestry are eligible, steps downstream of any capped step keep their forward-computed positions. This keeps each asset's work block compact rather than frontloading unconstrained steps against the chain start while capped production steps land weeks later.

Sweep-line craft-window placement

Craft occupancy is tracked as two sorted lists per craft: start dates and end dates. The overlap count for a proposed span `[s, s+D-1]` is computed in `O(log W)` via bisect:

```
overlap = bisect_right(starts, s+D-1) - bisect_left(ends, s)
```

When the proposed start is over-cap, candidate starts are `(window_end + 1)` for each existing window, the only dates where concurrency can drop. This eliminates the day-by-day stepping that made the original implementation `O(N2)` in the number of work items, reducing capped scheduling to `O(N log N)`.

AI Engine (`lib/scenario/generator.py`)

Uses Claude Sonnet. Three generation passes:

1. **Products pass:** generates the product list with names and dates from scope
2. **Asset profiles pass:** generates `{profiles: [{variable_values, total_count, priority}]}`; server-side distributes totals across products
3. **Work templates pass:** generates `{templates: [{match_when, step_names}]}`; server-side looks up `estimate_days` from the matrix and schedules

Variable values keys returned by the AI are normalized against the studio's actual field names (exact → case-insensitive → starts-with → contains) before validation, handling abbreviated field names.

Both engines use `lib/scenario/shared.py` for all DB writes, profile normalization, and date computation.

Session Stages

Stage	Description
<code>scoping</code>	Haiku is gathering scope parameters
<code>pending_generation</code>	Scope locked; awaiting generation worker pick-up
<code>generating</code>	Generation worker is running
<code>discussion</code>	Generation complete; Haiku can answer questions about the plan
<code>generation_failed</code>	Generation threw an exception; retryable

Ephemeral Data Tables

Scenario data is fully isolated from the canonical layer, no foreign keys to `canonical_assets` or any production tables. Dismissing a session cascades deletes through all five tables.

scenario_sessions

id	uuid	PK
studio_id	uuid	→ studios
user_id	uuid	→ auth.users
name	text	(NULL = ephemeral; reserved for future saved scenarios)
status	text	(active dismissed)
ai_stage	text	(scoping pending_generation generating discussion generated)
generation_mode	text	(ai rule_based)
scope_json	jsonb	(locked once pending_generation is set)
message_count	int	
preflight_warnings	jsonb	(array of {profile, issue, ...} objects)
created_at, expires_at		(24h TTL; nightly cleanup deletes expired rows)

scenario_messages

session_id	uuid	→ scenario_sessions (CASCADE)
studio_id	uuid	
role	text	(user assistant)
content	text	
created_at		

scenario_products

session_id	uuid	→ scenario_sessions (CASCADE)
studio_id	uuid	
name	text	
target_release_date	date	(NULL for earliest_ship until post-scheduling write-back)
created_at		

scenario_assets

session_id	uuid	→ scenario_sessions (CASCADE)
studio_id	uuid	
product_id	uuid	→ scenario_products (CASCADE)
name	text	
variable_values	jsonb	(must match estimate_config.variable_fields keys)
priority	text	
created_at		

scenario_work

session_id	uuid → scenario_sessions (CASCADE)
studio_id	uuid
asset_id	uuid → scenario_assets (CASCADE)
step_name	text
craft	text
estimate_days	numeric
start_date	date
end_date	date
created_at	

Wizard Mode (Rule-Based)

POST /api/scenario/generate is a direct-to-generation path: the wizard form (ScenarioWizard.jsx) collects scope parameters from the user directly (no chat), then posts a fully-formed scope body. Any active session for the user is dismissed first. The session is created in pending_generation immediately, bypassing the scoping stage.

GET /api/scenario/wizard-data returns the profiles and crafts the wizard needs to populate its dropdowns.

Frontend

ScenarioPlanner.jsx routes between three components based on session state:

- **No session** → ScenarioWizard , form-based scope input; direct generation path
- **Session with scoping/generating stage** → ScenarioChat , conversation interface
- **Session with discussion/failed stage** → ScenarioChat (discussion) + ScenarioViewer (data)

ScenarioViewer.jsx renders a three-tab read-only view of the generated plan: Products list, Assets list, and Work list with Gantt timeline view. All three datasets are paginated server-side via drain_pages so there is no display cap regardless of scenario size.

Work tab controls:

- **Filter bar:** filter by product (dropdown of all products in the scenario), craft (dropdown of all crafts), or asset name (free-text search). Active filters are highlighted; a "Clear" button resets all three. Tab label shows filtered/total count when any filter is active.
- **Group by:** groups work rows by Product, Asset, or Craft. Each group is collapsible. The column for the active group axis is hidden to avoid redundancy.
- **Table / Timeline toggle:** switches between the flat/grouped table and the Gantt timeline. Timeline is a per-asset swimlane chart with month headers, colour-coded bars by craft, and a legend. Group controls are hidden in timeline mode.

Generation mode badge: a small label in the tab bar shows `Rule-based` (highlighted) or `AI` to indicate which engine produced the scenario.

Deferred actions: "Export to CSV" and "Write to Source" buttons appear in the action bar but are disabled (grey). Both are planned future features.

API Reference

All endpoints are under `/api/scenario` . Studio-only.

Method	Path	Description
POST	<code>/start</code>	Create or resume active session
GET	<code>/wizard-data</code>	Profiles + crafts for wizard form
POST	<code>/generate</code>	Direct generation with fully-formed scope (wizard path)
POST	<code>/{{id}}/message</code>	Send a message in scoping or discussion stage
POST	<code>/{{id}}/force-generate</code>	Skip remaining scoping; generate with partial scope
POST	<code>/{{id}}/regenerate</code>	Apply <code>scope_changes</code> to stored scope and re-queue generation (discussion stage only)
POST	<code>/{{id}}/retry-generation</code>	Retry after <code>generation_failed</code>
GET	<code>/{{id}}/data</code>	Session state, scope, and all generated rows (paginated, no row limit)
GET	<code>/{{id}}/messages</code>	Full message history
DELETE	<code>/{{id}}</code>	Dismiss session

Known Gaps

Export and write-back deferred: "Export to CSV" and "Write to Source" in the action bar are disabled. Exporting the scenario plan to a spreadsheet and writing it back to the studio's source tool are both planned but not yet built.

No session history or comparison: each session produces a fresh plan with no link to prior scenarios. Comparing two plan alternatives requires running two sessions manually.

Discussion mode context on large scenarios: the Haiku discussion prompt uses a compact aggregated summary (product spans, profile distribution, craft utilisation, step counts). Per-asset and per-product detail is fetched on demand via tools. This design keeps the context bounded regardless of scenario size, but very large tool result payloads (e.g. `list_assets_in_product` on a product with hundreds of assets) may still approach token limits.

Missing `compare_profile_costs` discussion tool, Haiku cannot yet answer "which asset type is cheapest to produce?" or "how does 3D allocation differ between Hero and Support characters?" with grounded data. A `compare_profile_costs` tool is planned that aggregates `scenario_work` by profile label and returns ranked craft-day totals per profile.

Craft caps are greedy without backtracking: assets are placed in priority order and each asset claims the earliest available cap slot. A sub-optimal early placement can push later assets past their target window even when a valid schedule exists. The sweep-line placement ensures the earliest feasible slot for each step is found efficiently, but the overall asset ordering is not optimised.